

Bachelor-Thesis

zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

an der Hochschule für Technik und Wirtschaft des Saarlandes

im Studiengang Praktische Informatik

der Fakultät für Ingenieurwissenschaften

Netzwerk-Monitoring Dashboard mit Automatisierung

vorgelegt von

Xudong Zhang

betreut und begutachtet von

Prof. Dr.-Ing. André Miede

Andreas Schaffhauser

Saarbrücken, den 16. August 2026

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit (bei einer Gruppenarbeit: den entsprechend gekennzeichneten Anteil der Arbeit) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.

Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, den 16. August 2026

Xudong Zhang

Zusammenfassung

Kurze Zusammenfassung des Inhaltes in deutscher Sprache, der Umfang beträgt zwischen einer halben und einer ganzen DIN A4-Seite.

Orientieren Sie sich bei der Aufteilung bzw. dem Inhalt Ihrer Zusammenfassung an Kent Becks Artikel: <http://plg.uwaterloo.ca/~migod/research/beck00PSLA.html>.

*We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.*

— Donald E. Knuth [5]

Danksagung

Hier können Sie Personen danken, die zum Erfolg der Arbeit beigetragen haben, beispielsweise Ihren Betreuern in der Firma, Ihren Professoren/Dozenten an der htw saar, Freunden, Familie usw.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Zielsetzung	1
1.2	Problemstellung	2
1.3	Ziele der Arbeit	2
1.3.1	Pflichtziele	3
1.3.2	Optionale Erweiterungen	4
1.4	Geplantes Vorgehen	4
1.5	Aufbau der Arbeit	5
2	Grundlage	7
2.1	Grundbegriffe und Definitionen	7
2.1.1	Netzwerk-Monitoring	7
2.1.2	Verfügbarkeit und Erreichbarkeit	7
2.1.3	Latenz	7
2.1.4	Metriken, Exporter und Scraping	8
2.1.5	Alert und Alerting	8
2.1.6	Container und Container-Orchestrierung	8
2.1.7	Spring Boot, Actuator und Micrometer	9
2.1.8	TCP-basierte Erreichbarkeitsprüfung	9
2.2	Technologische Grundlagen	9
2.2.1	Architektur des Spring-Boot-basierten Monitoring-Dienstes	10
2.2.2	Micrometer und Prometheus-Integration	10
2.2.3	Visualisierung mit Grafana	11
2.2.4	Alerting mit Alertmanager	11
2.2.5	Containerisierte Bereitstellung mit Docker und Docker Compose	11
2.2.6	Netzwerkonnktivitätsprüfung mittels TCP-Sockets	12
2.3	Verwendete Methoden und Konzepte des entwickelten Systems	12
2.3.1	Konfigurationsgetriebenes Gerätemanagement und zeitgesteuertes Polling	12
2.3.2	Zustandsbasierte Metriken und Kennzahlenmodell	13
2.3.3	Schichtenarchitektur, Fehlerbehandlung und Robustheit	13
2.3.4	Automatisierte Tests der Kernlogik	14
2.4	Stand der Technik und Einordnung	14
3	Analyse	15
4	Konzeption	17
5	Implementierung	19
6	Evaluation	21
7	Zusammenfassung und Ausblick	23
	Literatur	25

Abbildungsverzeichnis	27
Tabellenverzeichnis	27
Listings	27
Abkürzungsverzeichnis	29
A Erster Abschnitt des Anhangs	33

1 Einleitung

1.1 Motivation und Zielsetzung

Die fortschreitende Digitalisierung betrieblicher Abläufe führt dazu, dass die Verfügbarkeit von Netzwerkdiensten und Anwendungen in modernen IT-Umgebungen eine zentrale Voraussetzung für einen stabilen Arbeitsbetrieb darstellt. Bereits kleinere Störungen in Netzwerkverbindungen oder auf Anwendungsebene können dazu führen, dass Arbeitsprozesse unterbrochen, Reaktionszeiten verlängert oder zentrale IT-Dienste nur eingeschränkt nutzbar werden. Vor diesem Hintergrund gewinnt ein systematisches und möglichst automatisiertes Monitoring zunehmend an Bedeutung.

Die Wahl des Themas „Netzwerk-Monitoring Dashboard mit Automatisierung“ ergibt sich aus mehreren fachlichen und praktischen Aspekten.

- **Praxisbezug aus dem Praktikum:** Im Rahmen des IT-Service-Praktikums beim Bildungscampus Saarland wurde deutlich, wie stark Netzwerkstörungen den laufenden Betrieb beeinträchtigen können. Dabei zeigte sich zugleich die Relevanz eines modernen und automatisierten Netzwerk-Monitorings.
- **Berufliche Relevanz:** Die angestrebte fachliche Ausrichtung liegt im Bereich DevOps und Netzwerkinfrastruktur. Monitoring stellt in diesem Umfeld eine zentrale Kompetenz dar und besitzt zugleich eine hohe Bedeutung auf dem aktuellen IT-Arbeitsmarkt.
- **Technisches Interesse:** Das Thema verbindet zentrale Interessensfelder der praktischen Informatik, insbesondere Netzwerkinfrastruktur, Automatisierung sowie moderne Werkzeuge des IT-Betriebs.

Der Praxisbezug des Themas ist besonders relevant, da Monitoring im IT-Service nicht nur der technischen Beobachtung von Systemen dient, sondern unmittelbar zur Sicherstellung betrieblicher Abläufe beiträgt. Werden Netzwerkprobleme oder Leistungsabfälle zu spät erkannt, können Helpdesk-Prozesse, Fernzugriffe, interne Verwaltungsanwendungen oder weitere digitale Dienste verzögert oder gestört werden. Ein automatisiertes Monitoring kann in solchen Situationen dazu beitragen, Störungen früher zu erkennen, Zustände transparenter darzustellen und die Reaktionsfähigkeit des IT-Betriebs zu verbessern.

Darüber hinaus besitzt das Thema auch aus fachlicher Sicht eine hohe Relevanz, da es mehrere Kernbereiche der praktischen Informatik miteinander verbindet. Dazu zählen unter anderem verteilte Systeme, Netzwerkinfrastruktur, automatisierte Datenerfassung, Zeitreihenverarbeitung, Visualisierung und einfache Alarmierungsmechanismen. Die Arbeit greift damit sowohl technische als auch organisatorische Fragestellungen auf und eignet sich daher besonders für eine praxisnahe Abschlussarbeit mit wissenschaftlichem Anspruch.

Das übergeordnete Ziel der Arbeit besteht darin, ein nachvollziehbares und technisch realisierbares Konzept für ein Monitoring-System zu entwickeln und dessen Umsetzbarkeit anhand eines Prototyps zu demonstrieren. Neben der rein technischen Realisierung soll auch gezeigt werden, wie sich verschiedene Werkzeuge sinnvoll kombinieren lassen,

1 Einleitung

um einen konsistenten Monitoring-Prozess von der Datenerfassung bis zur Visualisierung und Alarmierung abzubilden.

1.2 Problemstellung

In modernen IT-Infrastrukturen ist die kontinuierliche Überwachung von Netzwerkverbindungen, insbesondere im LAN- und VPN-Bereich, sowie des Zustands von Anwendungen von zentraler Bedeutung für einen stabilen und zuverlässigen Betrieb. Gerade in IT-Service-Umgebungen können Netzwerkstörungen oder Fehler auf Anwendungsebene in kurzer Zeit zu erheblichen betrieblichen Beeinträchtigungen führen. Dies gilt insbesondere dann, wenn mehrere Systeme voneinander abhängen und Störungen nicht isoliert, sondern kettenartig auf andere Dienste wirken.

In vielen IT-Service-Abteilungen werden Netzwerkprobleme jedoch weiterhin vor allem durch manuelle Ping-Tests oder unregelmäßige Einzelprüfungen erkannt. Solche Verfahren können zwar in einfachen Situationen kurzfristig hilfreich sein, sie sind jedoch nur bedingt geeignet, um größere oder dynamische Infrastrukturen zuverlässig zu überwachen. Die fehlende Automatisierung hat zur Folge, dass Störungen verspätet entdeckt werden, der manuelle Prüfaufwand hoch bleibt, historische Daten für weiterführende Analysen fehlen und bei Ausfällen keine unmittelbare Alarmierung erfolgt.

Diese Vorgehensweise ist nicht nur ineffizient, sondern auch fehleranfällig. Manuelle Kontrollen sind stark vom Zeitpunkt der Durchführung, vom Erfahrungswissen der Beteiligten und von der systematischen Dokumentation einzelner Beobachtungen abhängig. Dadurch entsteht das Risiko, dass kurzzeitige Ausfälle unbemerkt bleiben oder dass Zusammenhänge zwischen verschiedenen Fehlerbildern im Nachhinein nicht mehr zuverlässig rekonstruiert werden können.

Hinzu kommt, dass ohne kontinuierlich gespeicherte Messwerte nur eingeschränkt beurteilt werden kann, ob ein Problem einmalig aufgetreten ist oder ob sich bereits über längere Zeit ein negativer Trend abgezeichnet hat. Für einen professionellen IT-Betrieb ist jedoch nicht nur die unmittelbare Störungserkennung relevant, sondern auch die Möglichkeit, Entwicklungen über einen längeren Zeitraum nachzuvollziehen, Grenzwerte zu definieren und daraus gezielte Maßnahmen abzuleiten.

Ein professioneller IT-Betrieb erfordert daher ein Monitoring, das automatisiert, kontinuierlich und nachvollziehbar arbeitet. Es sollte Metriken auf unterschiedlichen Ebenen erfassen, diese zentral speichern, für operative und analytische Zwecke visualisieren und bei kritischen Zuständen eine zeitnahe Alarmierung ermöglichen. Genau an dieser Stelle setzt die vorliegende Arbeit an, indem ein prototypisches Monitoring-System für Netzwerk- und Anwendungsebene konzipiert und umgesetzt wird.

1.3 Ziele der Arbeit

Ziel der Arbeit ist die Entwicklung eines Monitoringsystems für LAN-/VPN-Verbindungen sowie für eine Spring-Boot-Anwendung. Das System soll Messdaten automatisiert erfassen, speichern, visualisieren und bei kritischen Zuständen eine Alarmierung auslösen. Damit verfolgt die Arbeit das Ziel, einen durchgängigen technischen Ablauf von der Generierung über die Erfassung und Speicherung bis hin zur Auswertung und Benachrichtigung abzubilden.

Die wesentlichen Zielsetzungen lauten wie folgt:

- Erfassung von Anwendungsmetriken durch eine Spring-Boot-Komponente mit Micrometer, insbesondere HTTP-Antwortzeiten, Fehlerquoten, Thread-Anzahl, JVM-

Speicher und CPU-Auslastung.

- Überwachung zusätzlicher Netzwerkmetriken wie Ping-Latenz, Erreichbarkeit, VPN-Status und Port-Verfügbarkeit ausgewählter Dienste.
- Regelmäßige Speicherung aller Metriken in Prometheus als Zeitreihendatenbasis zur Auswertung aktueller und historischer Werte.
- Visualisierung zentraler Kennzahlen aus Anwendung, Netzwerk und Host-Systemen in einem Grafana-Dashboard.
- Umsetzung einer einfachen Alarmierungsfunktion, die bei Ausfällen oder Schwellenwertüberschreitungen automatisiert eine E-Mail-Benachrichtigung versendet.

Die einzelnen Zielsetzungen greifen ineinander. Die Datenerfassung bildet die Grundlage für jede weitere Verarbeitung, da ohne verlässliche Metriken weder eine Zeitreihenspeicherung noch eine aussagekräftige Visualisierung möglich ist. Ebenso setzt eine funktionierende Alarmierung voraus, dass Zustandsänderungen und Grenzwertverletzungen zunächst systematisch erkannt und ausgewertet werden.

Aus fachlicher Sicht verfolgt die Arbeit damit nicht nur die Implementierung einzelner Werkzeuge, sondern die Entwicklung einer durchgängigen Monitoring-Kette. Entscheidend ist dabei, dass die verschiedenen Komponenten nicht isoliert betrachtet, sondern als zusammenhängendes System verstanden werden. Dadurch wird die Arbeit über eine reine Tool-Installation hinaus erweitert und erhält einen eigenständigen konzeptionellen Mehrwert.

Um den Arbeitsumfang innerhalb des vorgegebenen Bearbeitungszeitraums realistisch zu gestalten, wird zwischen Pflichtzielen und optionalen Erweiterungen unterschieden. Diese Trennung ist notwendig, um einen klaren Mindestumfang festzulegen und zugleich weiterführende Entwicklungsmöglichkeiten transparent zu machen.

1.3.1 Pflichtziele

Die Pflichtziele definieren den Kern der Arbeit und müssen im Rahmen des Projekts umgesetzt werden. Sie bilden zugleich die Grundlage für die spätere Bewertung des Prototyps.

- **Datenerfassung:** Entwicklung einer Spring-Boot-API mit Micrometer-Anbindung zur Erfassung von JVM-Metriken wie Speicher, CPU-Auslastung und Threads sowie von HTTP-Metriken wie Antwortzeit und Fehlerquote.
- **Speicherung:** Aufbau einer Prometheus-basierten Zeitreihenspeicherung mit definierten Scraping-Intervallen, beispielsweise 15 Sekunden, um aktuelle und historische Daten auswertbar zu machen.
- **Visualisierung:** Erstellung eines Grafana-Dashboards mit Panels für Ping-Latenz, VPN-Uptime, Port-Status und JVM-Leistung.
- **Alarmierung:** Implementierung eines Bash-Skripts, das bei Schwellenwertüberschreitungen, beispielsweise bei einer Ping-Latenz von über 200 ms oder einem VPN-Ausfall, automatisiert eine E-Mail-Benachrichtigung versendet.

Diese Pflichtziele decken die für das Vorhaben zentralen Funktionsebenen ab. Sie stellen sicher, dass sowohl auf Netzwerk- als auch auf Anwendungsebene relevante Zustände beobachtet werden können und dass die erhobenen Informationen nicht nur gesammelt,

1 Einleitung

sondern auch praktisch nutzbar gemacht werden. Die Kombination aus Datenerhebung, Speicherung, Visualisierung und Alarmierung bildet damit den funktionalen Kern des Prototyps.

1.3.2 Optionale Erweiterungen

Die optionalen Erweiterungen gehen über den definierten Mindestumfang hinaus und dienen dazu, mögliche Ausbaustufen des Systems aufzuzeigen. Sie sind insbesondere dann relevant, wenn einzelne Kernfunktionen früher als geplant umgesetzt werden oder wenn sich im Projektverlauf zusätzlicher Spielraum ergibt.

- **Ansible-Integration:** Automatisierte Bereitstellung der Monitoring-Infrastruktur auf mehreren Servern.
- **Zusätzliche Dashboards:** Getrennte Ansichten für Anwendungs-, Netzwerk- und Host-Ebene.
- **Erweiterte Alarmregeln:** Mehrkriterielle Alarmierung, etwa bei gleichzeitiger Prüfung von VPN-Status, Ping-Latenz und Port-Erreichbarkeit.
- **Rollen- und Rechte management:** Verwaltung von Benutzerrollen und Zugriffsrechten innerhalb des Grafana-Dashboards.

Die optionalen Erweiterungen verdeutlichen, dass das entworfene System grundsätzlich skalierbar und ausbaufähig angelegt ist. Gleichzeitig unterstreichen sie, dass die vorliegende Arbeit einen bewussten Fokus auf einen belastbaren Kernumfang legt, ohne weiterführende Entwicklungsmöglichkeiten auszuschließen.

1.4 Geplantes Vorgehen

Das Vorgehen orientiert sich an einem typischen Ablauf informatischer Abschlussarbeiten. Zunächst wird der fachliche und organisatorische Rahmen präzisiert, insbesondere im Hinblick auf die zu überwachenden Hosts, Netzwerkpfade und VPN-Verbindungen, geeignete Schwellenwerte sowie datenschutzbezogene Anforderungen. Diese Präzisierung ist notwendig, um ein realistisches und konsistentes Zielsystem für die weiteren Projektphasen festzulegen.

Anschließend erfolgt eine Recherche zu Spring Boot, Micrometer, Prometheus und Grafana, um die Eignung dieser Technologien für das geplante Monitoring-Szenario zu bewerten. Dabei geht es nicht nur um eine funktionale Betrachtung einzelner Werkzeuge, sondern auch um deren Zusammenspiel innerhalb einer Gesamtarchitektur. Die Recherche dient damit zugleich als Grundlage für spätere Technologieentscheidungen.

Darauf aufbauend wird ein Systementwurf entwickelt, der Architektur und Datenflüsse der Lösung beschreibt. Dieser Entwurf legt fest, welche Komponenten miteinander kommunizieren, wie Metriken erzeugt und übertragen werden und an welchen Stellen Visualisierung sowie Alarmierung ansetzen. Eine saubere konzeptionelle Vorarbeit ist wesentlich, damit die spätere Implementierung nicht nur funktioniert, sondern auch nachvollziehbar strukturiert ist.

Danach erfolgt die prototypische Implementierung, vorzugsweise in einer containerisierten Umgebung mit Docker. Die Containerisierung erleichtert eine reproduzierbare Bereitstellung der einzelnen Komponenten und unterstützt damit sowohl die technische

Umsetzung als auch die spätere Demonstration des Systems. In diesem Schritt wird geprüft, ob sich die zuvor entworfene Architektur mit den ausgewählten Technologien konsistent umsetzen lässt.

Abschließend wird das System anhand typischer Fehlerszenarien wie erhöhter Ping-Latenz, geschlossenen Ports oder eines VPN-Ausfalls getestet und hinsichtlich Erkennungszeit, Dashboard-Qualität und Alarmierungsfunktion bewertet. Die Evaluation dient nicht nur dazu, die Funktionalität des Prototyps nachzuweisen, sondern auch seine Grenzen sichtbar zu machen und mögliche Ansatzpunkte für spätere Erweiterungen zu identifizieren.

1.5 Aufbau der Arbeit

Nach der Einleitung werden zunächst die für das Verständnis der Arbeit erforderlichen fachlichen und technischen Grundlagen dargestellt. Anschließend folgt die Analyse des Problemraums, der Anforderungen sowie relevanter bestehender Ansätze.

Darauf aufbauend beschreibt die Arbeit die Konzeption des entwickelten Systems und erläutert anschließend dessen prototypische Implementierung. Abschließend erfolgt eine Evaluation der Lösung, bevor die zentralen Ergebnisse zusammengefasst und mögliche Erweiterungen im Ausblick dargestellt werden.

2 Grundlage

2.1 Grundbegriffe und Definitionen

Die folgenden Ausführungen führen die zentralen Begriffe ein, die für das Verständnis der im Rahmen dieser Arbeit entwickelten Monitoring-Lösung erforderlich sind. Sie schaffen ein einheitliches Begriffsverständnis als Grundlage für die nachfolgenden Kapitel zur Analyse, Konzeption und Implementierung.

2.1.1 Netzwerk-Monitoring

Unter Netzwerk-Monitoring wird die kontinuierliche Überwachung des Zustands und der Leistungsfähigkeit von Netzwerkkomponenten, Diensten und Schnittstellen verstanden. Dazu gehören beispielsweise Server, Kommunikationsdienste oder Webanwendungen, die über das Netzwerk erreichbar sein müssen. Monitoring-Systeme sammeln hierzu fortlaufend Daten über Verfügbarkeit, Antwortzeiten und Fehlersituationen, um Störungen frühzeitig zu erkennen und gezielt Gegenmaßnahmen einleiten zu können [2].

Im Kontext dieser Arbeit bezieht sich das Netzwerk-Monitoring vor allem auf die Prüfung der Erreichbarkeit von definierten Netzwerkzielen anhand von Host- und Portangaben. Als Beispiele gelten eine Spring-Boot-Anwendung oder die zugehörigen Monitoring-Komponenten. Die ermittelten Prüfergebnisse werden als Metriken exponiert und durch eine Monitoring- und Visualisierungsplattform gesammelt und visualisiert.

2.1.2 Verfügbarkeit und Erreichbarkeit

Verfügbarkeit beschreibt, in welchem Anteil einer betrachteten Zeitspanne ein System oder Dienst den vorgesehenen Zweck erfüllen kann. In der Praxis wird Verfügbarkeit häufig in Prozentwerten angegeben (z. B. „99,9 % Verfügbarkeit“ im Sinne eines Service Level Agreements). Für operative Monitoring-Lösungen ist jedoch weniger der abstrakte Prozentwert relevant, sondern die konkrete Erreichbarkeit zu einem bestimmten Zeitpunkt.

In dieser Arbeit wird die Erreichbarkeit eines Dienstes vereinfacht als binärer Zustand modelliert. Ein Ziel gilt als „up“, wenn innerhalb einer vorgegebenen Zeitspanne eine Verbindung aufgebaut werden kann, und als „down“, wenn der Verbindungsaufbau fehlschlägt oder die Zeitspanne überschritten wird. Dieser Zustand wird im entwickelten System in einer eigenen Datenstruktur sowie als Metrik `network_device_up` abgebildet, wobei der Wert 1,0 einem erreichbaren und 0,0 einem nicht erreichbaren Ziel entspricht.

2.1.3 Latenz

Neben der reinen Erreichbarkeit spielt die Latenz eine wesentliche Rolle bei der Beurteilung der Dienstqualität. Unter Latenz wird die Zeitspanne zwischen dem Senden einer Anfrage und dem Empfangen der zugehörigen Antwort verstanden. Im Netzwerkumfeld wird die Latenz üblicherweise in Millisekunden angegeben und dient als Kennzahl für die Reaktionsfähigkeit eines Dienstes.

Im Rahmen dieser Arbeit wird die Latenz für jeden Verbindungsversuch zu einem überwachten Ziel gemessen. Dazu wird die Zeitdifferenz zwischen Beginn und Ende des Verbindungsaufbaus berechnet. Das Ergebnis wird sowohl in der Datenstruktur `CheckResult` als auch in Form der Metrik `network_device_latency_ms` abgelegt. Damit steht für jedes überwachte Ziel neben dem binären Erreichbarkeitsstatus auch eine quantitative Aussage über die aktuelle Antwortzeit zur Verfügung.

2.1.4 Metriken, Exporter und Scraping

Eine Metrik ist eine messbare Kennzahl, die einen bestimmten Aspekt des Zustands oder der Performance eines Systems beschreibt. Typische Beispiele im Monitoring-Kontext sind CPU-Auslastung, Arbeitsspeicherkonsum, Anzahl von Anfragen pro Sekunde oder die bereits erläuterte Latenz. Moderne Monitoring-Lösungen arbeiten meist metrikbasiert, das heißt, sie erfassen systematisch Metriken, speichern diese in einer Zeitreihendatenbank und stellen sie für Auswertung und Alerting bereit [2].

Ein Exporter ist in diesem Zusammenhang eine Komponente, die Metriken in einem standardisierten Format bereitstellt, sodass sie von einem Monitoring-System wie Prometheus abgeholt werden können. Die Daten werden nicht aktiv an das Monitoring-System gesendet, sondern über einen HTTP-Endpunkt bereitgestellt.

Prometheus folgt einem sogenannten Pull- bzw. Scraping-Modell: Das System ruft in festen Intervallen die konfigurierten Metrik-Endpunkte der überwachten Anwendungen ab und speichert die dort gelieferten Kennzahlen [1]. In der hier entwickelten Lösung stellt die Spring-Boot-Anwendung über den Actuator-Endpunkt `/actuator/prometheus` einen Exporter bereit, der die Anwendungsmesswerte im Prometheus-Format exponiert [3].

2.1.5 Alert und Alerting

Ein Alert ist eine automatisch generierte Meldung, die auftritt, wenn eine oder mehrere Bedingungen erfüllt sind, die zuvor als kritisch oder dringend definiert wurden. Alerting bezeichnet den gesamten Prozess von der Auswertung von Metriken über die Erkennung solcher Bedingungen bis hin zur Benachrichtigung von zuständigen Personen oder Systemen. Ziel ist es, relevante Abweichungen möglichst frühzeitig und zuverlässig zu melden, ohne gleichzeitig eine Flut irrelevanter Meldungen zu erzeugen.

Im Prometheus-Ökosystem wird Alerting typischerweise über Alert-Regeln realisiert, die auf den gespeicherten Metriken operieren. Sobald eine Regel erfüllt ist, erzeugt Prometheus eine Alert-Nachricht und übergibt sie an den Alertmanager, der für die weitere Verarbeitung, Gruppierung und Weiterleitung zuständig ist [2]. In der im Rahmen dieser Arbeit verwendeten Konfiguration sind unter anderem Regeln definiert, die ausgelöst werden, wenn ein überwachter Dienst nicht erreichbar ist („`NetworkDeviceDown`“) oder die Latenz einen vordefinierten Grenzwert überschreitet („`HighNetworkLatency`“).

2.1.6 Container und Container-Orchestrierung

Container sind leichtgewichtige, isolierte Laufzeitumgebungen, in denen Anwendungen samt ihrer Abhängigkeiten ausgeführt werden. Im Gegensatz zu klassischen virtuellen Maschinen teilen sich Container den Kernel des Host-Betriebssystems, was sie ressourceneffizient und schnell startbar macht. Ein Container-Image beschreibt dabei den unveränderlichen Ausgangszustand eines Containers, einschließlich Laufzeitumgebung und Anwendung.

Container-Orchestrierung bezeichnet die koordinierte Verwaltung mehrerer Container, einschließlich Start, Stopp, Vernetzung und Konfigurationsverwaltung. Im Kontext dieser Arbeit kommt Docker als Container-Plattform zum Einsatz, sowie Docker Compose zur Orchestrierung der beteiligten Dienste. Die zentrale Monitoring-Anwendung, Prometheus, Grafana und der Alertmanager werden in jeweils eigenen Containern betrieben und über ein gemeinsames virtuelles Netzwerk verbunden. Die notwendigen Einstellungen sind in der Datei `docker-compose.yml` definiert.

2.1.7 Spring Boot, Actuator und Micrometer

Spring Boot ist ein Framework zur vereinfachten Entwicklung von Java-basierten Anwendungen. Es stellt umfangreiche Auto-Konfiguration und eine integrierte Projektstruktur bereit, wodurch typische Infrastrukturaufgaben reduziert werden. Für produktionsreife Anwendungen bietet Spring Boot mit dem Actuator-Modul eine Sammlung von Betriebseindpunkten, die Informationen über den Zustand und die Metriken der Anwendung zur Verfügung stellen [9].

Micrometer ist eine Bibliothek zur Erfassung und Bereitstellung von Metriken in Java-Anwendungen. Sie fungiert als Abstraktionsschicht, die die Anwendung von der konkreten Wahl des Monitoring-Backends entkoppelt und unterstützt unter anderem Prometheus, Graphite und andere Systeme [9]. In der hier verwendeten Projektkonfiguration wird Micrometer zusammen mit dem Prometheus-Registry-Modul eingebunden, sodass die Anwendung automatisch einen Prometheus-kompatiblen Metrik-Endpunkt bereitstellt. Die Grundkonfiguration erfolgt über die Anwendungskonfigurationsdatei, in der die Exposition von Actuator-Endpunkten wie `/actuator/prometheus` gesteuert wird [3].

2.1.8 TCP-basierte Erreichbarkeitsprüfung

Das Transmission Control Protocol (TCP) ist ein verbindungsorientiertes Transportprotokoll, das eine zuverlässige Datenübertragung zwischen zwei Endpunkten bereitstellt. Bevor Daten übertragen werden können, wird eine Verbindung zwischen Client und Server aufgebaut. Gelingt dieser Verbindungsaufbau, ist der entsprechende Dienst an der angegebenen Adresse erreichbar; schlägt er fehl, ist entweder der Dienst nicht verfügbar oder es liegt eine Netzwerkstörung vor.

Die im Rahmen dieser Arbeit implementierte Erreichbarkeitsprüfung basiert auf dem Versuch, eine TCP-Verbindung zu einem definierten Host und Port innerhalb eines vorgegebenen Zeitlimits aufzubauen. Technisch wird dazu ein Socket verwendet, der versucht, die Verbindung zu initiieren. Wird der Verbindungsaufbau innerhalb des Timeouts erfolgreich abgeschlossen, wird das Ziel als erreichbar („up“) eingestuft; bei Ausnahmefällen oder Zeitüberschreitungen wird das Ziel als nicht erreichbar („down“) markiert. Die dabei gemessene Dauer des Verbindungsversuchs dient gleichzeitig als Grundlage für die Latenzmetrik.

2.2 Technologische Grundlagen

Die entwickelte Monitoring-Lösung basiert auf mehreren miteinander verbundenen Technologien. Im Folgenden werden Spring Boot, Micrometer, Prometheus, Grafana, Alertmanager sowie Docker und Docker Compose als technologische Grundlage des Systems erläutert.

2.2.1 Architektur des Spring-Boot-basierten Monitoring-Dienstes

Der zentrale Bestandteil der Lösung ist ein auf Spring Boot basierender Monitoring-Dienst. Spring Boot stellt einen geeigneten Rahmen für die Entwicklung von Java-Anwendungen bereit, da es die Konfiguration typischer Komponenten vereinfacht und die Entwicklung von Web- und Betriebsfunktionen unterstützt. Der Dienst stellt REST-Schnittstellen zur Abfrage von Geräten und Prüfergebnissen zur Verfügung. Darüber hinaus werden über Spring Boot Actuator Betriebsinformationen und Metriken für externe Monitoring-Systeme bereitgestellt [9].

Die Architektur des Dienstes folgt einer klaren funktionalen Trennung. Die Konfigurationskomponente liest die zu überwachenden Ziele sowie Zeitüberschreitungen aus der Anwendungskonfiguration ein. Das Datenmodell bildet Netzwerkziele und Ergebnisse einzelner Prüfungen ab. Die fachliche Prüf- und Metriklogik ist in einer Service-Schicht gekapselt, während ein Scheduler die Überprüfungen in festen Intervallen ausführt. Controller stellen die Ergebnisse über HTTP-Endpunkte bereit. Diese Aufgabentrennung verbessert die Wartbarkeit und ermöglicht eine gezielte automatisierte Prüfung einzelner Komponenten.

Die zu überwachenden Ziele werden durch Name, Host, Port und Aktivierungsstatus beschrieben. Für jedes aktivierte Ziel führt der Dienst eine Erreichbarkeitsprüfung durch und erzeugt daraus einen Statuswert sowie eine gemessene Verbindungsdauer. Die zuletzt ermittelten Ergebnisse können über eine REST-Schnittstelle abgerufen werden. Zusätzlich kann eine Prüfung bei Bedarf manuell ausgelöst werden. Dadurch ist der Dienst sowohl für die automatische Überwachung als auch für eine direkte technische Kontrolle geeignet.

2.2.2 Micrometer und Prometheus-Integration

Micrometer dient als Metrik-Fassade innerhalb der Spring-Boot-Anwendung. Die Bibliothek erlaubt es, Messwerte unabhängig von einem konkreten Monitoring-Backend zu erfassen. Spring Boot Actuator unterstützt Micrometer durch automatische Konfiguration und kann Metriken für verschiedene Monitoring-Systeme bereitstellen [9].

Für die entwickelte Lösung wird das Prometheus-Registry-Modul verwendet. Dadurch werden die erfassten Werte in einem Format ausgegeben, das Prometheus verarbeiten kann. Spring Boot stellt hierfür den Endpunkt `/actuator/prometheus` bereit. Dieser Endpunkt liefert eine textbasierte Darstellung der verfügbaren Metriken und kann von einem Prometheus-Server regelmäßig abgefragt werden [9].

Die Anwendung erzeugt für jedes überwachte Netzwerkziel zwei zentrale Metriken. Die Metrik `network_device_up` repräsentiert die Erreichbarkeit eines Ziels als numerischen Zustand. Dabei steht der Wert 1 für ein erreichbares und der Wert 0 für ein nicht erreichbares Ziel. Die Metrik `network_device_latency_ms` enthält die zuletzt gemessene Dauer eines Verbindungsversuchs in Millisekunden. Beide Metriken werden mit den Labels `device` und `host` versehen, sodass die Werte einzelnen Netzwerkzielen zugeordnet werden können.

Prometheus arbeitet nach einem Pull-Modell. In festgelegten Intervallen ruft der Prometheus-Server Metriken von konfigurierten Targets ab. Die Konfiguration umfasst dabei insbesondere die Adresse des Ziels, den Abrufpfad sowie das Abrufintervall. In der vorliegenden Lösung ist die Spring-Boot-Anwendung als Target konfiguriert; Prometheus fragt den Pfad `/actuator/prometheus` der Anwendung ab. Die erfassten Werte werden als Zeitreihen gespeichert und stehen anschließend für Abfragen, Visualisierungen und Alert-Regeln zur Verfügung [8].

2.2.3 Visualisierung mit Grafana

Grafana ist eine Plattform zur Visualisierung und Analyse von Monitoring-Daten. Sie kann Prometheus als Datenquelle verwenden und ermöglicht die Erstellung von Dashboards, die Zeitreihen, Tabellen oder Statusanzeigen darstellen. Prometheus fungiert dabei als Datenlieferant, während Grafana die gespeicherten Metriken über Abfragen abrufen und grafisch aufbereitet [4].

Für die Monitoring-Lösung wird Grafana als eigenständiger Container betrieben. Die Verbindung zu Prometheus wird mittels einer Provisioning-Konfiguration automatisch eingerichtet. Grafana unterstützt die Definition von Datenquellen über YAML-Dateien; bei Verwendung von Docker Compose kann hierfür der Name des Prometheus-Containers als Hostname verwendet werden [4]. Dadurch ist nach dem Start der Containerumgebung keine manuelle Konfiguration der Datenquelle erforderlich.

Auf einem Dashboard können beispielsweise die aktuelle Verfügbarkeit der Ziele sowie ihre Latenz dargestellt werden. Die Verfügbarkeitsmetrik eignet sich für Statusanzeigen, während die Latenzmetrik als Zeitreihe dargestellt werden kann. Dadurch lassen sich kurzfristige Ausfälle und längerfristige Veränderungen der Antwortzeiten erkennen. Die Visualisierung ergänzt damit die maschinelle Auswertung durch Prometheus um eine für Administratorinnen und Administratoren verständliche Darstellung.

2.2.4 Alerting mit Alertmanager

Die Darstellung von Messwerten in einem Dashboard allein reicht nicht aus, um zeitkritische Störungen zuverlässig zu behandeln. Daher ergänzt ein Alerting-System die Überwachung. Prometheus wertet hierfür definierte Alert-Regeln aus. Eine Regel beschreibt eine Bedingung, bei deren länger andauernder Erfüllung ein Alert ausgelöst wird. Typische Bedingungen sind die Nichterreichbarkeit eines Dienstes oder das Überschreiten eines festgelegten Latenzgrenzwertes.

Für das Routing und die Verarbeitung ausgelöster Alerts wird Alertmanager eingesetzt. Alertmanager nimmt Alerts von Prometheus entgegen und übernimmt deren Deduplizierung, Gruppierung und Weiterleitung an konfigurierte Empfänger, beispielsweise E-Mail-, Webhook- oder Incident-Management-Systeme [7]. Dadurch lassen sich mehrere gleichartige Meldungen zusammenfassen und unnötige Benachrichtigungen reduzieren.

Die Einrichtung des Alertings besteht aus drei wesentlichen Schritten: der Konfiguration des Alertmanagers, der Verbindung von Prometheus mit Alertmanager sowie der Definition von Alert-Regeln in Prometheus [6]. Im entwickelten System überwacht eine Regel den Status `network_device_up`. Liegt dieser für ein Ziel über einen definierten Zeitraum bei 0, wird der Alert „NetworkDeviceDown“ ausgelöst. Eine weitere Regel prüft, ob `network_device_latency_ms` einen Schwellenwert überschreitet. Dadurch können sowohl vollständige Ausfälle als auch auffällige Verzögerungen erfasst werden.

2.2.5 Containerisierte Bereitstellung mit Docker und Docker Compose

Die einzelnen Bestandteile der Monitoring-Lösung werden containerisiert bereitgestellt. Docker ermöglicht die Ausführung von Anwendungen in isolierten Containern, die alle erforderlichen Laufzeitabhängigkeiten enthalten. Dadurch wird die Bereitstellung von Anwendungen unabhängig von der konkreten Umgebung besser reproduzierbar. Für die Java-Anwendung wird ein Docker-Image verwendet, das die zuvor erzeugte ausführbare JAR-Datei enthält und diese über die Java-Laufzeit startet.

Docker Compose dient zur Beschreibung und zum gemeinsamen Start mehrerer zusammengehöriger Container. In der vorliegenden Lösung werden der Monitoring-Dienst, Pro-

metheus, Grafana und Alertmanager als getrennte Services definiert. Die Services befinden sich in einem gemeinsamen Docker-Netzwerk und können daher über ihre Servicennamen miteinander kommunizieren. Beispielsweise erreicht Prometheus den Monitoring-Dienst über den Hostnamen `app`, während Grafana Prometheus über den Hostnamen `prometheus` ansprechen kann.

Neben der Vernetzung werden auch Abhängigkeiten und persistente Speicherbereiche definiert. Der Start von Prometheus erfolgt erst nach dem Start der Anwendung; Grafana wird anschließend nach Prometheus gestartet. Für Prometheus, Grafana und Alertmanager werden Volumes verwendet, um Konfigurationen oder dauerhaft gespeicherte Daten nicht beim Neustart eines Containers zu verlieren. Die zentrale Bereitstellung über Docker Compose ermöglicht somit einen reproduzierbaren und übersichtlichen Betrieb des gesamten Monitoring-Stacks.

2.2.6 Netzwerkkonnektivitätsprüfung mittels TCP-Sockets

Die Erreichbarkeit der überwachten Ziele wird durch einen TCP-basierten Verbindungsversuch geprüft. TCP ist ein verbindungsorientiertes Transportprotokoll. Bei einem erfolgreichen Verbindungsaufbau kann davon ausgegangen werden, dass der angegebene Host erreichbar ist und ein Dienst auf dem konfigurierten Port Verbindungen akzeptiert. Diese Art der Prüfung ist aussagekräftiger als eine reine Überprüfung der IP-Erreichbarkeit, da sie zugleich einen konkreten Dienstport berücksichtigt.

Der Monitoring-Dienst erstellt für jedes Ziel einen Socket und versucht, innerhalb eines konfigurierbaren Timeouts eine Verbindung zum jeweiligen Host und Port aufzubauen. Wird die Verbindung erfolgreich hergestellt, wird das Ziel als erreichbar markiert. Tritt beim Verbindungsaufbau eine Ausnahme auf oder wird das Timeout überschritten, erhält das Ziel den Status „down“. Der Fehler führt dabei nicht zum Abbruch des gesamten Prüfzyklus, sondern wird für das betroffene Ziel als Prüfergebnis verarbeitet.

Zusätzlich wird die Dauer des Verbindungsversuchs gemessen. Diese Zeitspanne wird als Latenzwert gespeichert und als Metrik exportiert. Dabei ist zu beachten, dass der gemessene Wert die Dauer des TCP-Verbindungsaufbaus beschreibt. Er stellt daher keine vollständige Ende-zu-Ende-Antwortzeit einer Anwendung dar, sondern eine Kennzahl für die Erreichbarkeit und den Aufbau einer Transportverbindung. Diese Einschränkung wird bei der späteren Bewertung der Messergebnisse berücksichtigt.

2.3 Verwendete Methoden und Konzepte des entwickelten Systems

Die entwickelte Monitoring-Lösung folgt mehreren methodischen und architektonischen Prinzipien. Im Mittelpunkt stehen eine konfigurationsbasierte Verwaltung der Überwachungsziele, die regelmäßige Überprüfung ihrer Erreichbarkeit sowie die Bereitstellung der Ergebnisse als Metriken. Ergänzend wurden eine klar getrennte Schichtenarchitektur, eine robuste Fehlerbehandlung und automatisierte Tests für zentrale Funktionen umgesetzt.

2.3.1 Konfigurationsgetriebenes Gerätemanagement und zeitgesteuertes Polling

Die zu überwachenden Netzwerkziele werden nicht fest im Quellcode hinterlegt, sondern über die Anwendungskonfiguration definiert. Zu jedem Ziel werden ein Name, ein Host,

ein Port sowie ein Aktivierungsstatus angegeben. Zusätzlich werden zentrale Parameter wie das Prüfintervall und das Verbindungs-Timeout konfiguriert.

Die Konfiguration wird beim Start der Anwendung in eine strukturierte Konfigurationsklasse übernommen. Dadurch wird die technische Beschreibung der Überwachungsziele von der eigentlichen Prüfungslogik getrennt. Neue Ziele können durch eine Anpassung der Konfigurationsdatei ergänzt werden, ohne dass Änderungen an den Java-Klassen erforderlich sind. Der Aktivierungsstatus ermöglicht außerdem, einzelne Ziele bei Bedarf vorübergehend von der Überwachung auszunehmen.

Die Überwachung erfolgt nach dem Prinzip des zeitgesteuerten Pollings. Dabei überprüft der Monitoring-Dienst alle aktivierten Ziele in regelmäßigen Abständen. Für die Umsetzung wird die Scheduling-Funktion von Spring Boot verwendet. Ein Scheduler löst die zentrale Prüfmethode nach Ablauf eines konfigurierbaren Intervalls aus. In der vorliegenden Konfiguration beträgt das Standardintervall 15 Sekunden.

Das Polling-Verfahren stellt einen Kompromiss zwischen Aktualität und Ressourcenverbrauch dar. Kürzere Intervalle ermöglichen eine schnellere Erkennung von Störungen, verursachen jedoch mehr Verbindungsversuche und Metrikaktualisierungen. Längere Intervalle reduzieren den Aufwand, führen jedoch zu einer größeren Verzögerung bei der Erkennung eines Ausfalls. Durch die Konfigurierbarkeit des Intervalls kann dieser Kompromiss an die jeweilige Einsatzumgebung angepasst werden.

2.3.2 Zustandsbasierte Metriken und Kennzahlenmodell

Die Ergebnisse der Erreichbarkeitsprüfungen werden als zustandsbasierte Metriken modelliert. Für jedes überwachte Ziel werden zwei Kennzahlen bereitgestellt: ein Verfügbarkeitsstatus und eine Latenzmetrik. Der Status wird durch die Metrik `network_device_up` dargestellt. Ein Wert von 1 bedeutet, dass der Verbindungsaufbau erfolgreich war; ein Wert von 0 kennzeichnet ein nicht erreichbares Ziel.

Die zweite Metrik `network_device_latency_ms` enthält die Dauer des letzten Verbindungsversuchs in Millisekunden. Beide Werte werden als Gauges exportiert. Ein Gauge ist für diese Art von Information geeignet, da er einen aktuellen Zustand beschreibt und sich sowohl erhöhen als auch verringern kann. Dies unterscheidet ihn beispielsweise von einem Counter, der ausschließlich monoton ansteigt.

Zur Unterscheidung der einzelnen Ziele werden die Metriken mit den Labels `device` und `host` versehen. Dadurch können Prometheus-Abfragen, Alert-Regeln und Grafana-Dashboards den Status und die Latenz eines konkreten Ziels auswerten. Das Kennzahlenmodell bildet damit die Grundlage für die Visualisierung und die automatische Erkennung von Störungen.

2.3.3 Schichtenarchitektur, Fehlerbehandlung und Robustheit

Die Anwendung ist in mehrere fachlich getrennte Komponenten gegliedert. Diese Struktur folgt dem Prinzip der Schichtenarchitektur und unterstützt eine klare Zuordnung von Verantwortlichkeiten. Die Konfigurationskomponente übernimmt das Einlesen und Bereitstellen der Einstellungen aus der Datei `application.yml`. Das Datenmodell umfasst die Klassen `Device` und `CheckResult`. Während `Device` ein zu überwachendes Ziel beschreibt, enthält `CheckResult` das Ergebnis einer konkreten Prüfung, einschließlich Erreichbarkeitsstatus, Latenz und Prüfzeitpunkt.

Die zentrale fachliche Logik befindet sich im `DeviceCheckService`. Dieser Dienst führt die Prüfungen aus, aktualisiert die Metrikwerte und hält die zuletzt ermittelten Prüfergebnisse bereit. Der `MonitoringScheduler` stößt die periodische Ausführung an. Die Controller-Schicht stellt REST-Endpunkte bereit, über die Geräte, aktuelle Prüfergebnisse

2 Grundlage

und der Status des Monitoring-Dienstes abgefragt werden können. Zusätzlich ist eine manuelle Auslösung eines Prüfzyklus vorgesehen.

Netzwerkprüfungen sind fehleranfällig, da Verbindungsprobleme, nicht verfügbare Dienste, ungültige Zieladressen oder Zeitüberschreitungen auftreten können. Deshalb werden Ausnahmen beim Aufbau einer TCP-Verbindung behandelt. Schlägt ein Verbindungsaufbau fehl, wird das betreffende Ziel als nicht erreichbar eingestuft; die Prüfung der weiteren Ziele wird dennoch fortgesetzt.

Ein konfigurierbares Timeout begrenzt die Wartezeit für jeden Verbindungsversuch. Dadurch wird verhindert, dass nicht reagierende Ziele den Prüfzyklus über einen längeren Zeitraum blockieren. Auch bei einem fehlgeschlagenen Verbindungsversuch werden Status und Dauer der Prüfung als Ergebnis gespeichert. Die Kombination aus Fehlerbehandlung, Timeout und unabhängiger Verarbeitung einzelner Ziele trägt zur Robustheit der Monitoring-Lösung bei.

2.3.4 Automatisierte Tests der Kernlogik

Zur Absicherung zentraler Funktionen wurden automatisierte Unit-Tests umgesetzt. Der Schwerpunkt liegt auf dem `DeviceCheckService`, da dieser die Konfiguration der Ziele verarbeitet und die Ergebnisse der Erreichbarkeitsprüfungen erzeugt.

Ein Test prüft, ob die in der Konfiguration definierten Geräte korrekt in die interne Geräteliste übernommen werden. Dadurch wird sichergestellt, dass die konfigurationsgetriebene Verwaltung der Ziele wie vorgesehen funktioniert. Ein weiterer Test verwendet ein nicht erreichbares Ziel und prüft, ob dieses als nicht erreichbar bewertet wird. Zusätzlich wird geprüft, ob für den Verbindungsversuch ein gültiger Latenzwert erzeugt wird.

Für die Tests werden JUnit 5 sowie eine einfache Micrometer-Meter-Registry verwendet. Die Tests ermöglichen eine wiederholbare Überprüfung der Kernlogik, ohne dass dafür die vollständige Containerumgebung mit Prometheus, Grafana und Alertmanager gestartet werden muss. Sie reduzieren damit das Risiko, dass Änderungen an der Anwendung unbemerkt zu Fehlern in der grundlegenden Monitoring-Funktion führen.

2.4 Stand der Technik und Einordnung

Blackbox Exporter???

3 Analyse

4 Konzeption

5 Implementierung

6 Evaluation

7 Zusammenfassung und Ausblick

Literatur

- [1] Baeldung. *Monitor a Spring Boot App Using Prometheus*. <https://www.baeldung.com/spring-boot-prometheus> (Zugriff am 13.08.2026). 2024.
- [2] Brian Brazil. *Prometheus: Up & Running*. O'Reilly, 2019.
- [3] Grafana Labs. *Configure Spring Boot to Generate Prometheus Metrics*. <https://grafana.com/docs/grafana-cloud/platform/knowledge-graph/advanced-configuration/enable-prom-metrics-collection/application-frameworks/springboot/> (Zugriff am 13.08.2026). 2024.
- [4] Grafana Labs. *Configure the Prometheus Data Source*. <https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/> (Zugriff am 15.08.2026). 2026.
- [5] Donald E. Knuth. „Computer Programming as an Art“. In: *Communications of the ACM* 17.12 (1974), S. 667–673.
- [6] Prometheus Authors. *Alerting Overview*. <https://prometheus.io/docs/alerting/latest/overview/> (Zugriff am 15.08.2026). 2026.
- [7] Prometheus Authors. *Alertmanager*. <https://prometheus.io/docs/alerting/latest/alertmanager/> (Zugriff am 15.08.2026). 2026.
- [8] Prometheus Authors. *Configuration*. <https://prometheus.io/docs/prometheus/latest/configuration/configuration/> (Zugriff am 15.08.2026). 2026.
- [9] Spring Team. *Metrics :: Spring Boot*. <https://docs.spring.io/spring-boot/reference/actuator/metrics.html> (Zugriff am 13.08.2026). 2024.

Abbildungsverzeichnis

Tabellenverzeichnis

Listings

Abkürzungsverzeichnis

Anhang

A Erster Abschnitt des Anhangs

In den Anhang gehören „Hintergrundinformationen“, also weiterführende Information, ausführliche Listings, Graphen, Diagramme oder Tabellen, die den Haupttext mit detaillierten Informationen ergänzen.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kolophon

Dieses Dokument wurde mit der L^AT_EX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt